LAB 15

Backtest-Engine Stage: Cost Models, Execution Timing and Break-Even

Item	Detail
Topic	Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis
Pipeline stage	Backtest engine
TSC reference	A7, A8, K7
Duration	25 min
Folder	labs_fullcode/Lab15-trading-costs/
Files	Lab15-trading-costs/backtest_stage.py, pipeline/backtest.py, common/backtest.py, Lab15-trading-costs/solution/backtest.py
Prompts	Lab15_Prompts.pdf in this folder (step -> prompt, ready to copy)
Mode	Runs ONLINE with your Alpaca paper keys (.env), or OFFLINE on the bundled data and the paper-broker simulator. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

Alpaca advertises commission-free trading and a trader concludes that costs do not matter. Before the pipeline is automated, the head of execution wants the backtest engine to show gross versus net results under four cost scenarios, where the money goes, the break-even cost of a fast and a slow strategy, and how sensitive results are to the assumed fill time.

Objective

Build pipeline/backtest.py around the common backtester: run any library signal under every cost scenario (and a custom one), break the cost into commission, spread, slippage and regulatory fees, find each strategy's break-even cost and test next-open versus next-close execution.

Before You Start

- Open a terminal in the labs_fullcode folder, activate your virtual environment and open your AI coding assistant there (set-up below). The complete, working code is already in this folder.
- Record every result you are asked to check - the same evidence is asked for in the Practical Performance assessment.

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 15	python3 Lab15-trading-costs/backtest_stage.py (same as: python -m pipeline backtest)
The complete code	pipeline/backtest.py (reference copy: Lab15-trading-costs/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab15-trading-costs/Lab15_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Read the cost model (K7)

Open common/backtest.py and read CostModel and COST_PRESETS (zero, alpaca_retail, per_share_broker, high_cost). Note that regulatory fees apply to sells only and that a trade costs half the spread plus slippage.

AI prompt - paste into your AI coding assistant
Explain CostModel.trade_cost in labs_fullcode/common/backtest.py with a worked example: a USD 95,000 buy and the matching sell of SPY at 676.64 under the alpaca_retail preset. Show each component in dollars. Do not change the file.

Check: A worked example where spread and slippage dominate and commission is zero.

Step 2: Build the engine helpers (A7)

Ask your assistant for the engine part of pipeline/backtest.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/backtest.py. run_backtest(bars, name, costs=None, execution='next_open', **params) = common.backtest.backtest(bars, pipeline.signals.target_position(name, bars, **params), costs, execution) - the single entry point later stages use. break_even_bps(bars, signal, lo=0, hi=1000): bisection (30 steps) on a one-way cost expressed as CostModel(slippage_bps=x) until net CAGR = 0 (return 0 if gross CAGR <= 0). parse_custom(['spread_bps=3', 'slippage_bps=3']) -> CostModel('Custom route', ...). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: break_even_bps on dip_buy returns about 19.5.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Build the scenario report (A7, A8)

Ask your assistant for run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/backtest.py add run(strategies=None, symbol='SPY', custom=None) for `python -m pipeline backtest --strategies --symbol --custom`: default strategies dip_buy and trend_filter; scenarios from config costs.scenarios plus the custom one; for each strategy and scenario print gross CAGR, net CAGR, cost drag, net Sharpe, executions and dollars of commission, spread, slippage, reg fees and total (from BacktestResult.cost_summary()); print each strategy's break-even bps per side next to its number of round trips and average gross trade in bps; print next-bar OPEN versus next-bar CLOSE CAGR and Sharpe with alpaca_retail costs; save _outputs/Lab15_cost_scenarios.csv and chart Lab15_costs.png (equity by scenario for the first strategy with the gross curve dashed, and a stacked bar of where the costs go). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline backtest --help shows --strategies, --symbol and --custom.

Step 4: Run the cost scenarios (A7, A8)

Run: python -m pipeline backtest --strategies dip_buy trend_filter (same as: python3 Lab15-trading-costs/backtest_stage.py).

Check: dip_buy: gross CAGR 5.09%; net 4.65% (Alpaca), 4.21% (per-share), 0.47% (high-cost, USD 33,560 of costs); 203 executions. trend_filter: gross 8.80%; net 8.70% / 8.59% / 7.68%; 47 executions.

Step 5: Analyse the cost breakdown (K7)

For the Alpaca scenario, which component is largest even though commission is zero?

Check: dip_buy with Alpaca costs: commission USD 0, spread 1,127, slippage 2,254, reg fees 317 - total USD 3,698; slippage and spread scale with turnover.

Step 6: Interpret break-even and execution timing (A8, K7)

Compare each break-even cost with the average trade, then read the execution-timing lines.

Check: Break-even: dip_buy 19.5 bps per side (102 round trips, average gross trade 41.7 bps) versus trend_filter 142.3 bps (24 round trips, 347.3 bps). Timing: dip_buy 4.65% at the next OPEN but 7.22% at the next CLOSE - the fill assumption alone moves the result; trend_filter 8.70% vs 8.66%.

Step 7: Configure your own execution route (A7)

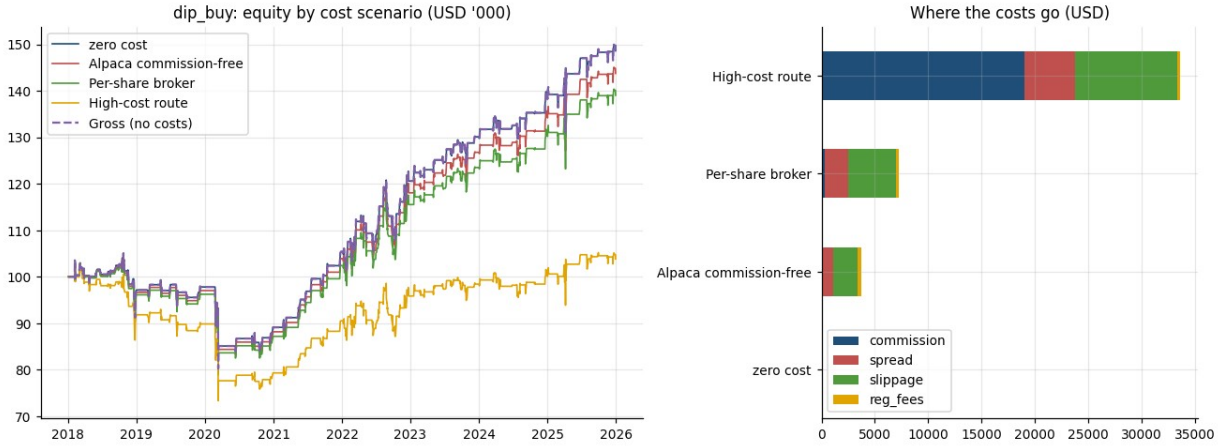
Run: python -m pipeline backtest --custom spread_bps=3 slippage_bps=3 and record the new net CAGR for both strategies.

Check: Custom route: dip_buy net 3.89% (USD 9,833 of costs); trend_filter 8.51% (USD 2,445).

Expected Output (Full Code)

python -m pipeline backtest: dip_buy gross CAGR 5.09%, net 4.65% (Alpaca) / 4.21% (per-share) / 0.47% (high-cost, USD 33,560 of costs); trend_filter gross 8.80%, net 8.70% / 8.59% / 7.68%; break-even 19.5 bps per side for dip_buy (102 round trips, average trade 41.7 bps) versus 142.3 bps for trend_filter; next-close fills lift dip_buy to 7.22%; custom route 3.89% / 8.51%; files _outputs/Lab15_cost_scenarios.csv and chart _outputs/Lab15_costs.png.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.



Lab 15 output (labs_fullcode/_outputs/Lab15_costs.png), offline mode on the bundled data.

Debrief

Commission is only one trading cost (K7): spread and slippage scale with turnover, and the assumed fill time can matter as much as the fee schedule. Coding the cost model into the engine (A7) and testing outcomes under several scenarios (A8) shows whether an edge survives real-world execution - the break-even cost is the single number that says how much room the strategy has.

Troubleshooting

Symptom	What to do
Net CAGR equals gross CAGR	You are using the zero preset; check costs.preset and costs.scenarios in config.yaml.
Regulatory fees look tiny	They are: a few dollars per million of sell notional. Rates change - check the current SEC and FINRA rates.
Break-even hits the upper bound	Increase hi in break_even_bps() for very slow strategies.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.